```
    rooms: list[RoomSchema] = Field(...,
        alias="room_set")

    ...
```

To nest the room data, use a list of RoomSchema type hint, which, when combined with the Field class, allows you to rename it from room_set.

Ninja's pretty cool; with very little code, you've got nested data. Rerun your query on the CBGB venue, and you'll now get the following:

```
{
    "id": 1,
    "name": "CBGB",
    "description": "Take it CJ! 1-2-3-4",
    "slug": "cbgb-1",
    "url": "/api/v1/bands/venue/1/",
    "rooms": [
        {
            "id": 2,
            "name": "Blue"
        },
        {
            "id": 1,
            "name": "Red"
        }
    ]
}
```

Nested room data associated with CBGBs

11.2.5 Authenticating your API

So far, you've only been doing the *R* from *CRUD*. What about the other three choices? You've used Ninja's `.get()` decorator to read values. It also has `.post()`, `.put()`, `.patch()`, and `.delete()` decorators to take care of the other parts of CRUD (still a fun acronym). But you might not want just anybody adding stuff to your database. Before considering create or delete, it is time for a tangent about authentication.

Ninja supports a variety of authentication mechanisms. You can specify authentication globally through the `NinjaAPI` object, at the `Router` level, or endpoint by endpoint. In each case, you provide an `auth` argument, the value of which determines the kind of authentication to perform. In this section, I'll be discussing two mechanisms: Django's auth and an API key. For a full list of the other supported types, see https://django-ninja.dev/guides/authentication/.

Authentication for an API doesn't have to be restricted to modification. Everything discussed here can also be used for your GET methods if you want to limit who can access your data.

What kind of authentication to use depends on your use case. As Django has authentication built-in, at first glance, that might seem like the right answer. If you've already got user accounts and all that fun password management stuff covered in chapter 6, why not use it for your API as well? The problem is the authentication step: if you're not authenticated, Django redirects you to the login page. If another system is talking to your project, that means it has to fill in a web form.